Telegram + Gemini Bot

Step-by-step setup guide for Windows and Android (Termux)

What you will build

A Telegram bot that answers your messages with Google Gemini. It picks a light model for easy questions and a stronger model for hard ones, falls back to the other model if one fails, and shows a "Thinking..." message while it works. You first test it on your Windows laptop, then run it 24/7 on a spare Android phone.

You need	Details
Telegram account	To create the bot and chat with it
Google account	To create a Gemini API key in Google AI Studio
Windows laptop	Python 3.10 or newer installed
Android phone	Termux installed from F-Droid (not the Play Store)

Part 1. Get your two keys

Step 1. Telegram bot token

- In Telegram, open **@BotFather** and send **/newbot**.
- Choose a display name, then a username that ends in **bot**.
- BotFather replies with a **token** like 123456789:ABC... Copy it somewhere private.

Step 2. Gemini API key

- Open **Google AI Studio** in your browser and sign in.
- Choose **Get API key** and create a key. Copy it somewhere private.

Warning: Treat both keys like passwords. Never paste them into chats, screenshots, or public folders. If a key is ever exposed, create a new one: **/revoke** in @BotFather for Telegram, and delete and recreate the Gemini key.

Part 2. Windows laptop (testing)

Step 1. Check Python

```
python --version
```

It should print Python 3.10 or higher (for example Python 3.11.x). If not, install Python from python.org and tick "Add Python to PATH" during setup.

Step 2. Create the project folder

Use a name without spaces. A space in a folder name needs quotes in every command.

```
mkdir F:\Project\phone-ai
cd F:\Project\phone-ai
```

If your folder has a space (for example F:\Telegram project), always write: **cd "F:\Telegram project"**

Step 3. Install the packages

```
python -m pip install "python-telegram-bot>=22" requests python-dotenv
python -m pip install --upgrade "apscheduler>=3.10.4,<3.12" tzlocal
```

Always use **python -m pip** so the packages go into the same Python that runs your bot. The second line prevents the "Only timezones from the pytz library" error.

Step 4. Create the .env file (your keys)

Run these two commands inside the project folder. First replace the two placeholder values with your real keys:

```
Set-Content -Path .env -Value "GEMINI_API_KEY=PASTE_REAL_GEMINI_KEY" -Encoding ascii
Add-Content -Path .env -Value "TELEGRAM_TOKEN=PASTE_REAL_TELEGRAM_TOKEN" -Encoding ascii
dir -Force
```

dir -Force must list a file named exactly **.env** (not .env.txt). The format inside is NAME=value with no quotes and no spaces around the equals sign.

> **Warning:** If the bot later says the token is **your-telegram-token** or **PASTE_REAL_TELEGRAM_TOKEN**, the placeholder was never replaced.

Step 5. Create bot.py

Easiest and safest: copy the file **bot_requests_version.py** (given with this guide) into the project folder and rename it to **bot.py**. Or type it in Notepad:

```
notepad bot.py
```

- Choose Yes if Notepad asks to create the file.
- Paste the full script from Part 4. If the file already had text, press CTRL+A and Delete first, so the new code does not land in the middle of the old text.
- Save with CTRL+S, and make sure the file is named bot.py, not bot.py.txt.

Step 6. Run the bot

```
cd F:\Project\phone-ai
python bot.py
```

Expected output:

```
Bot is starting...
Polling Telegram... waiting for messages.
```

Step 7. Test it

- Open your bot in Telegram and send a message such as "Hello".
- A "Thinking..." message appears immediately, then turns into the answer.
- In PowerShell you will see lines like Received, status 200, and Reply sent via ...
- To stop the bot press CTRL+C. To stop every Python program: **Stop-Process -Name python, pythonw -Force -ErrorAction SilentlyContinue**

> **Warning:** Only one copy of the bot can run with the same token. Stop the laptop copy before you start the phone copy, or you will get a Conflict error.

Part 3. Android phone with Termux (24/7)

Step 1. Install Termux

- Install **F-Droid**, then install **Termux** from it. The Play Store version is outdated and breaks package installs.

Step 2. Update and install Python

```
pkg update && pkg upgrade
pkg install python
python --version
```

Step 3. Install the packages

```
pip install "python-telegram-bot>=22" requests python-dotenv
```

Step 4. Allow storage access (one time)

```
termux-setup-storage
```

Tap **Allow** on the permission popup. If it asks "Do you want to continue? (y/n)", type **y** once and press Enter. Afterwards **ls ~/storage/downloads** should list your phone's Download folder (note: downloads, with an s).

Step 5. Create the project folder

```
mkdir -p ~/phone-ai
cd ~/phone-ai
pwd
```

Keep your files here, in Termux's own home folder, not in Downloads. Other apps can read Downloads, so your keys would not be private there.

Step 6. Create the .env file

Replace the placeholders with your real keys. The first line uses one > and the second uses two >>:

```
echo "GEMINI_API_KEY=PASTE_REAL_GEMINI_KEY" > .env
echo "TELEGRAM_TOKEN=PASTE_REAL_TELEGRAM_TOKEN" >> .env
ls -a
```

ls -a must show **.env**. (Files starting with a dot are hidden unless you use -a.)

Step 7. Put bot.py on the phone

Method A (most reliable): send the file. Send bot_requests_version.py to yourself in Telegram (Saved Messages), download it on the phone so it lands in Downloads, then copy it:

```
cp ~/storage/downloads/bot_requests_version.py ~/phone-ai/bot.py
ls -a
```

Method B: paste the code with a heredoc. Everything between the first line and the last line EOF is saved:

```
cd ~/phone-ai
cat > bot.py << 'EOF'
...paste the full script from Part 4 here...
EOF
```

Method C: the nano editor. Run **nano bot.py**, paste the code, then save with CTRL+X, Y, Enter. (CTRL is a button in Termux's extra keys row.)

> **Warning:** Pasting can remove the double underscores in __name__ and __main__. After creating the file, check it with: **grep -n "main" bot.py**. The last lines must read: if __name__ == "__main__":

Step 8. Run the bot

```
cd ~/phone-ai
termux-wake-lock
python bot.py
```

You should see "Bot is starting..." and "Polling Telegram... waiting for messages." Then message your bot on Telegram.

Step 9. Keep it running

Run it inside a loop so it restarts by itself if it crashes:

```
cd ~/phone-ai
termux-wake-lock
while true; do python bot.py; echo "Restarting in 5s..."; sleep 5; done
```

- Leave the phone on a charger.
- Android settings: Apps, Termux, Battery, set to Unrestricted (turn off battery optimization).
- Keep Wi-Fi on while the screen is off, and do not swipe Termux away from recent apps.
- To stop: tap CTRL and press C (twice if you used the loop).
- If the phone restarts or Termux closes, paste the start command again.
- When the internet drops the bot pauses, then reconnects by itself.

Step 10. Change a key later

```
cd ~/phone-ai
echo "GEMINI_API_KEY=NEW_GEMINI_KEY" > .env
echo "TELEGRAM_TOKEN=NEW_TELEGRAM_TOKEN" >> .env
```

Then restart the bot.

Part 4. The complete script (bot.py)

The same file works on Windows and in Termux. It uses the requests library, so there is nothing hard to install on the phone. The router picks **gemini-3.1-flash-lite** for easy questions and **gemini-3.8-flash** for hard ones; change the two MODEL lines near the top to use other models.

```
import os
import asyncio

import requests
from dotenv import load_dotenv
from telegram import Update
from telegram.ext import ApplicationBuilder, ContextTypes, MessageHandler, filters

load_dotenv()

GEMINI_API_KEY = os.environ["GEMINI_API_KEY"]
TELEGRAM_TOKEN = os.environ["TELEGRAM_TOKEN"]

MODEL_LITE = "gemini-3.1-flash-lite"
MODEL_PRO = "gemini-3.8-flash"
GEMINI_URL = "https://generativelanguage.googleapis.com/v1beta/models/{}:generateContent"


def call_gemini(model, prompt):
    headers = {"x-goog-api-key": GEMINI_API_KEY, "Content-Type": "application/json"}
    payload = {"contents": [{"parts": [{"text": prompt}]}]}
    r = requests.post(GEMINI_URL.format(model), headers=headers, json=payload, timeout=60)
    print(f"[{model}] status:", r.status_code)
    data = r.json()
    if r.status_code != 200:
        raise RuntimeError(f"{r.status_code}: {data.get('error', {}).get('message', 'error')}")
    return data["candidates"][0]["content"]["parts"][0]["text"]


def heuristic_check(prompt):
    lower = prompt.lower()
    if any(k in lower for k in ["debug", "refactor", "code architecture", "proof"]) or len(prompt) > 400:
        return MODEL_PRO
    words = lower.split()
    if len(words) <= 4 and words[0] in ("hi", "hello", "hey", "thanks", "ok"):
        return MODEL_LITE
```

```
    return None


def llm_classify(prompt):
    routing_prompt = (
        "Classify the following query difficulty as either 'EASY' (casual chit-chat, "
        "simple question, translation) or 'HARD' (math, coding, logical reasoning, "
        "multi-step planning, detailed analysis).\n"
        "Output ONLY the single word EASY or HARD.\n\n"
        f"Query: {prompt}"
    )
    try:
        if "HARD" in call_gemini(MODEL_LITE, routing_prompt).upper():
            return MODEL_PRO
    except Exception:
        pass
    return MODEL_LITE


def execute_with_fallback(primary, prompt):
    fallback = MODEL_LITE if primary == MODEL_PRO else MODEL_PRO
    try:
        return call_gemini(primary, prompt), primary
    except Exception as e:
        print(f"[!] {primary} failed ({e}). Falling back to {fallback}...")
        return call_gemini(fallback, prompt), f"{fallback} (fallback)"


async def handle_message(update: Update, context: ContextTypes.DEFAULT_TYPE):
    user_text = update.message.text or ""
    if not user_text.strip():
        await update.message.reply_text("Please send a text message.")
        return

    print(f"Received: {user_text}")
    status = await update.message.reply_text("Thinking...")
    try:
        model = heuristic_check(user_text)
        if not model:
            model = await asyncio.to_thread(llm_classify, user_text)
        reply, used = await asyncio.to_thread(execute_with_fallback, model, user_text)
        final_text = f"{reply}\n\n(Handled by: {used})"
        await status.edit_text(final_text[:4000])
        for i in range(4000, len(final_text), 4000):
            await update.message.reply_text(final_text[i:i + 4000])
        print(f"Reply sent via {used}")
    except Exception as e:
        print("Error occurred:", e)
        await status.edit_text("Sorry, something went wrong.")


if __name__ == "__main__":
    print("Bot is starting...")
    app = ApplicationBuilder().token(TELEGRAM_TOKEN).build()
    app.add_handler(MessageHandler(filters.TEXT & ~filters.COMMAND, handle_message))
    print("Polling Telegram... waiting for messages.")
    app.run_polling()
```

Part 5. Command cheat sheet

Task	Windows PowerShell	Termux (phone)
Show current folder	pwd	pwd
Go into project	cd F:\Project\phone-ai	cd ~/phone-ai
Go up one folder	cd ..	cd ..
List files (with hidden)	dir -Force	ls -a
Make a folder	mkdir name	mkdir -p name
Edit a file	notepad bot.py	nano bot.py
Show a file	type bot.py	cat bot.py
Install packages	python -m pip install ...	pip install ...
Run the bot	python bot.py	python bot.py
Stop the bot	CTRL+C	CTRL (button) then C
Keep awake	-	termux-wake-lock
Check library version	python -m pip show python-telegram-bot	pip show python-telegram-bot

Part 6. Troubleshooting

What you see	Cause and fix
KeyError: 'GEMINI_API_KEY'	.env missing, misnamed (must be exactly .env), or you are in the wrong folder. Run dir -Force or ls -a.
InvalidToken ... your-telegram-token	The placeholder text is still in .env. Put the real token in with the Step 4 / Step 6 commands.
Conflict: terminated by other getUpdates	Two copies use the same token. Stop the other copy (laptop or phone), or /revoke for a new token.
ImportError: cannot import name 'Application'	Old Telegram library. Run python -m pip uninstall -y python-telegram-bot, then install "python-telegram-bot>=22".
Only timezones from the pytz library	Old apscheduler. Run python -m pip install --upgrade "apscheduler>=3.10.4,<3.12" tzlocal
name 'file' is not defined / name 'name' is not defined	Double underscores were lost while pasting. Copy the file instead of pasting, or fix __file__ / __name__.
IndentationError or SyntaxError	Code was pasted into the middle of old text or lost its spaces. Empty the file (CTRL+A, Delete) and paste once, or copy the ready-made file.
No module named 'google'	Only the google-genai version needs it: python -m pip install google-genai. The script in this guide does not.
cd : A positional parameter cannot be found	The folder name has a space. Use quotes: cd "F:\Telegram project"
Status 503 (high demand)	Google is busy. The bot tries the other model automatically; if both fail, wait and retry.
Status 404 for a model	The model name is wrong or retired. Edit the two MODEL lines in bot.py.
Status 400 or 403	Gemini key is wrong or revoked. Create a new key.
Status 429	Free quota used up. Wait a while.
No reply and nothing printed	You may be messaging a different bot than the token belongs to.

Part 7. Security checklist

- Keys live only in the .env file, never inside bot.py.
- Never share or upload .env. If you use Git, add a .gitignore file containing .env.
- Back up bot.py, but not .env. You can always create new keys.
- Never send screenshots or photos that show your keys. Rotate any key that was exposed.
- Use one bot token per running copy, and a separate folder, .env, and token for each project.